

In the Name of God



Faculty of Computer Engineering

Compiler Design

Spring Semester 1404–1405

Instructor: Dr. Alaeiyan

Head TA: Alireza Ghorbani

Final Project

Submission Deadline: -

This project is dedicated to the design and implementation of a **code-aware IDE feature set**¹ for a programming language chosen by each team. The system must be capable of receiving source code in the target language, analyzing it according to formally defined rules, and producing enriched, interactive output at three increasingly sophisticated levels: visual (syntax coloring), semantic (scope and type understanding), and structural (program-wide analysis and refactoring).

The project sits at the intersection of three bodies of knowledge that are central to this course:

- **Formal Language Theory:** regular languages, context-free grammars, and pushdown automata.
- **Compiler Front-End Design:** lexing, parsing, AST construction, symbol tables, and type systems.
- **Program Analysis:** control-flow graphs, call graphs, data-flow analysis, and static error detection.

The implemented system must include the following core components:

- A **Lexer** (Lexical Analyzer) for converting raw character streams into typed token sequences.
- A **Parser** (Syntactic Analyzer) for reconstructing the hierarchical program structure from tokens.
- A **Grammar Specification** in BNF or EBNF notation, fully documented.

¹Well-known production-grade examples include Pygments (highlighting), clangd (C/C++ language server), Pyright (Python type checker), and rust-analyzer — all built on the same theoretical foundations you study here.

- A **Semantic Analyzer** for scope resolution, symbol binding, and type checking.
- A **Program Analysis** module for call graphs, control-flow graphs, and IDE navigation features.

Why This Project Matters for Compiler Design

Every modern compiler — GCC, Clang, javac, rustc — begins exactly here: a lexer tokenizes the source file into tokens, a parser assembles those tokens into a syntax tree according to a formal grammar, a semantic pass resolves names and types, and analysis passes extract information for optimization and code generation. This project builds a complete *compiler front-end* and extends it with the analysis passes that power modern IDEs. By the end, you will have hand-crafted the core pipeline that every production language tool is built upon. The data structures and algorithms you implement here — DFAs, recursive-descent parsers, symbol tables, CFGs — are not abstract academic concepts; they are running right now inside every IDE on every developer's machine.

1. Project Goals and Motivation

- Gain hands-on experience with automata, regular expressions, and formal grammars by building a complete, working Lexer and Parser from scratch.
- Understand and implement the full compilation pipeline: from raw characters all the way to structured, semantically annotated program representations.
- Build modular, extensible software that supports multiple target languages without requiring changes to the shared core architecture.
- Implement scope resolution and a type system, understanding the theoretical jump from context-free to context-sensitive language properties that this requires.
- Construct program analysis infrastructure (CFG, call graphs, data-flow) that mirrors what production compilers build for optimization and verification.
- Produce a usable, practical tool that could serve as a plugin for a real text editor or as a standalone educational programming environment.

2. Language Selection

Teams are encouraged to choose a language they are already comfortable reading, since the focus should be on the compiler design concepts rather than on learning the language syntax during the project.

3. System Architecture Overview

The system follows the classical **compiler front-end pipeline**, extended with IDE-specific analysis passes. The overall data flow is illustrated below.

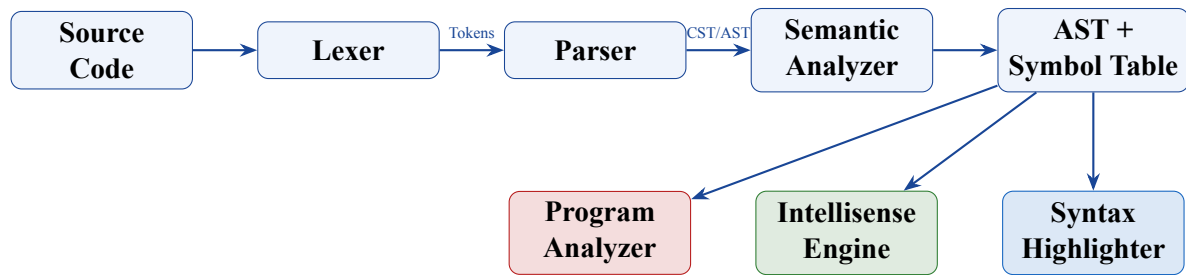


Figure 1: System pipeline from source code to IDE features.

Component Responsibilities

- **Lexer**: converts a character stream into a typed token sequence. Handles whitespace, comments, string escapes, and reports lexical errors.
- **Parser**: consumes the token stream and builds a **Concrete Syntax Tree** (CST) or directly an **Abstract Syntax Tree** (AST) according to the formal grammar.
- **Semantic Analyzer**: walks the AST to build the **Symbol Table**, resolve name bindings, perform type checking, and annotate AST nodes with type and scope information.
- **Syntax Highlighter** (Phase 1): traverses the annotated AST and maps each node to a color category.
- **Intellisense Engine** (Phase 2): queries the Symbol Table and AST to produce completion lists, hover info, and diagnostics.
- **Program Analyzer** (Phase 3): constructs CFGs and call graphs, and implements navigation and refactoring operations.

4. Phase One: Lexical Analysis and Syntax Highlighting

Phase 1 — Lexical Analysis & Syntax Highlighting

In Phase 1, teams implement the first two stages of the compiler front-end: the **Lexer** and a basic **Parser**, and then use the resulting token stream and AST to produce a **syntax highlighter**. The output is a correctly colored rendering of the source code, available both in a terminal (ANSI) and in a browser (HTML/CSS).

4.1. Theoretical Background: Regular Languages and DFAs

Compiler Design Connection — Lexical Analysis

The Lexer is the only compiler phase rooted entirely in **regular language theory**. Each token type is described by a regular expression. The union of all token regexes is converted — via Thompson’s construction — into an NFA, then — via the subset construction — into a **deterministic finite automaton (DFA)**, which is then minimized using Hopcroft’s algorithm and simulated on the input character stream. This is precisely what `lex` and `flex` do internally. In your implementation you may use a hand-written DFA, a regex library, or a table-driven approach, but you *must* document the formal regular expressions for every token class and explain how they compose into a single DFA with priority rules (longest-match, keyword-before-identifier).

4.2. Lexer Design and Implementation

The Lexer converts a **raw character stream** into a **token stream**. Each produced token must carry:

- A **type** (keyword, identifier, integer literal, string literal, operator, delimiter, comment, ...)
- A **lexeme**: the exact matched substring from the source.
- A **source location**: file name, line number, and column number — essential for later error reporting.

4.2.1. Required Token Categories

The following categories must be supported for any reasonable target language:

Category	Examples	Formal Description
Keywords	<code>if, while, return</code>	Finite set; checked before identifiers
Identifiers	<code>myVar, _count, x1</code>	<code>[a-zA-Z_] [a-zA-Z0-9_]*</code>
Integer literals	<code>42, 0xFF, 0b1010</code>	Decimal / hexadecimal / binary
Float literals	<code>3.14, 1.0e-5, .5f</code>	With optional exponent and suffix
String literals	<code>"hello\n"</code>	Quoted, with escape sequence handling
Character literals	<code>'a', '\t'</code>	Single character or escape
Operators	<code>+, ->, <=, ::</code>	Full language-specific set
Delimiters	<code>{, (, ;, ,</code>	Structural punctuation
Single-line comments	<code>// text</code>	From <code>//</code> to end-of-line
Block comments	<code>/* text */</code>	Possibly nested, depending on language
Whitespace	<code>spaces, tabs, newlines</code>	Tracked for location; usually discarded
Preprocessor directives	<code>#include, #define</code>	For C/C++; treated as a token class

4.2.2. Longest-Match and Priority Rules

When two rules can match at the current position, the Lexer must apply:

1. **Longest match (maximal munch):** always consume the longest possible token. For example, `<=` must be scanned as one token, not `<` followed by `=`.
2. **Priority:** keywords take priority over identifiers. The string `while` must produce `KEYWORD(while)`, not `IDENT(while)`.

4.2.3. Error Handling in the Lexer

The Lexer must **never crash** on invalid input. Instead:

1. Emit an `INVALID` token for any unrecognized character, recording its exact location.
2. **Recover** by advancing past the offending character and continuing.
3. Detect and report **unterminated string literals** and **unterminated block comments**.

Example: Token Stream for a C snippet

```

1 int factorial(int n) {
2     if (n <= 1) return 1;
3     return n * factorial(n - 1);
4 }

```

The Lexer produces (abbreviated):

```

KW(int) IDENT(factorial) LPAREN KW(int) IDENT(n) RPAREN LBRACE
KW(if) LPAREN IDENT(n) OP(<=) INT(1) RPAREN KW(return) INT(1) SEMI
KW(return) IDENT(n) OP(*) IDENT(factorial) LPAREN IDENT(n) OP(-) INT(1)
RPAREN SEMI
RBRACE EOF

```

Example: Lexer Error Recovery

```

1 int x@ = 5;      /* '@' is not a valid C token:
2                  INVALID('@') emitted at 1:6, then scan resumes */
3 int y = 10;      /* scanned correctly: INT(10) */

```

4.3. Parser Design and Grammar Specification**Compiler Design Connection — Parsing and CFGs**

The Parser corresponds to the study of **Context-Free Grammars (CFGs)** and **Push-down Automata (PDAs)**. A CFG defines the set of syntactically valid programs; a parser is an algorithm that decides membership in this language while simultaneously constructing a derivation tree. You must select and document your parsing strategy:

- **LL(1) / LL(k)** — top-down, implemented as a hand-written *recursive-descent parser*. Each non-terminal in the grammar becomes one function. This is the most common approach for hand-written parsers (used in Clang, GCC, and rustc).
- **LR(1) / LALR(1)** — bottom-up, implemented as a shift-reduce parser driven by a parse table (as produced by yacc/bison).

Whichever strategy you choose, you must demonstrate that your grammar is free of ambiguity (for LL parsers: no left recursion, no FIRST/FOLLOW conflicts for $k = 1$).

4.3.1. Grammar Documentation (Required Deliverable)

Teams must write a complete grammar for their chosen language in **EBNF** notation. The grammar must cover all statement and expression forms that appear in the test inputs. A representative fragment for C is shown below.

```

program      ::= declaration* EOF
declaration  ::= function_decl | var_decl | struct_decl
function_decl ::= type_spec IDENT '(' param_list? ')' block
param_list   ::= param (',' param)*
param        ::= type_spec IDENT
type_spec    ::= ('int' | 'float' | 'char' | 'void' | 'double') '*'
block        ::= '{' statement* '}'
statement    ::= if_stmt | while_stmt | for_stmt | return_stmt

```

```

| expr_stmt | block | var_decl
if_stmt      ::= 'if' '(' expr ')' statement ('else' statement)?
while_stmt   ::= 'while' '(' expr ')' statement
for_stmt     ::= 'for' '(' expr_stmt expr_stmt expr? ')' statement
return_stmt  ::= 'return' expr? ';'
expr_stmt    ::= expr? ';'
expr         ::= assignment
assignment   ::= IDENT ('='|'+='|'-='|'*=') assignment | logical_or
logical_or   ::= logical_and ('||' logical_and)*
logical_and  ::= equality ('&&' equality)*
equality     ::= relational (('=='|'!=') relational)*
relational   ::= additive (('<'|'>'|'<='|'>=') additive)*
additive     ::= multiplicative (('+'|'-') multiplicative)*
multiplicative ::= unary (('*'|'|'/'|'%'') unary)*
unary        ::= ('-'|'!'|'&'|'*') unary | postfix
postfix      ::= primary ('[' expr ']' | '(' arg_list? ')') | '.' IDENT |
    '->' IDENT)*
primary      ::= INT | FLOAT | STRING | CHAR | IDENT | '(' expr ')'
arg_list     ::= expr (',' expr)*

```

Listing 1: EBNF fragment for C function definitions and statements

Left Recursion Elimination

The grammar shown above is already left-recursion-free and suitable for recursive descent. For example, additive uses right-associative iteration (*) rather than left-recursive self-reference. If your language's grammar has left recursion, you must eliminate it before implementing the recursive-descent parser.

4.3.2. Abstract Syntax Tree (AST) Design

The Parser produces an **AST** in which:

- **Leaf nodes** represent terminals: literals and identifiers.
- **Internal nodes** represent grammatical constructs: BinaryExpr, IfStmt, FuncDecl, CallExpr, ReturnStmt, etc.
- Every node carries its **source location** (line + column of its first token) for error messages and IDE navigation.
- Nodes carry a **type annotation field** (initially null) to be filled in by the Semantic Analyzer.

Example: AST for return n * factorial(n - 1);

```

ReturnStmt
  value: BinaryExpr(op='*')
    left: Identifier(name='n', loc=3:12)
    right: CallExpr(callee='factorial', loc=3:16)
      args[0]: BinaryExpr(op='-')
        left: Identifier(name='n', loc=3:26)
        right: IntLiteral(value=1, loc=3:30)

```

4.3.3. Error Recovery in the Parser

The parser must survive syntax errors using **panic-mode recovery**: when an unexpected token is encountered, skip tokens until a *synchronization point* is found (a semicolon, closing brace, or keyword such as if or return), emit a helpful error message, then resume parsing.

Example: Parser Error Recovery

```

1  int x = ;          /* Error at 1:9: expected expression, got ';'
2                        Recovery: skip to ';', continue */
3  int y = 42;        /* Successfully parsed despite the error above */
4  if (y > 0          /* Error: missing ')' before '{' --- recovered */
5  {
6      return y;
7  }

```

4.4. Syntax Highlighting Rules

Using the annotated AST (or token stream for simpler cases), apply consistent color coding. The mapping below is a **minimum requirement**; teams may extend it.

Token / AST Node Category	Suggested Color	Rationale
Keywords (if, return, ...)	Bold blue	Primary structural markers
Type names (int, float, ...)	Teal / cyan	Type-level annotations
Variable identifiers	White / default	Neutral base
Function / method names	Yellow / gold	Callable distinction
Type / class names	Bright green	Type-level entities
Integer and float literals	Orange	Numeric values
String and char literals	Warm green	Text data
Boolean literals	Orange (same as numeric)	Constant values
Operators	Light gray	Low visual weight
Comments	Dim gray, italic	Non-executable
Preprocessor / decorator / annotation	Magenta	Meta-level constructs
Errors / invalid tokens	Red underline	Immediate visual feedback

AST-Level Highlighting is Required

Token-stream-only highlighting cannot distinguish a function-call identifier from a variable identifier. Your highlighter *must* query AST node types (and in later phases, the Symbol Table) to achieve accurate, context-aware coloring. A pure regex-based highlighter does **not** satisfy the requirements of this project.

4.5. Output Formats

1. **ANSI Terminal:** inject ANSI escape codes into the output (e.g., `\e[34;1m` for bold blue). Output must be a faithful rendering of the original source with colors injected around each token.
2. **HTML/CSS:** produce a self-contained HTML file with a `<pre>` block containing `<span class="kw">`, `<span class="lit">`, etc. elements, with a linked or embedded CSS stylesheet. Must render correctly in any modern browser without JavaScript.

4.6. Evaluation Criteria for Phase One

5. Phase Two: Semantic Analysis and Intellisense

Phase 2 — Semantic Analysis & Intellisense Engine

Phase 2 extends the system beyond syntactic analysis into **semantic understanding** of the program. The core deliverables are: a working **Symbol Table** with full scope resolution, a **type system** with checking or inference, and an **Intellisense engine** providing scope-aware auto-completion, hover information, and structured diagnostics. This phase corresponds to the *semantic analysis pass* in a production compiler.

Compiler Design Connection — Semantic Analysis

Semantic analysis is the compiler phase that checks *context-sensitive* properties — things that cannot be expressed in a CFG alone. This maps directly to the study of **attribute grammars** and **context-sensitive languages**. The Symbol Table is the central data structure, analogous to the *environment* in a lambda-calculus evaluator. Name resolution implements **lexical scoping**, and type checking implements a subset of a formal type system. For statically typed languages, this is **bidirectional typing**; for dynamically typed languages, it is **flow-sensitive type inference**. Both are active research topics rooted in the programming language theory you study in this course.

5.1. Symbol Table Construction

The **Symbol Table** is populated during a traversal of the AST. Each entry records:

Field	Description
name	The identifier string
kind	variable, function, type, parameter, class, field, method
type	The declared or inferred type (as a type expression)
scope	Reference to the enclosing scope node
definition_loc	File, line, column of the declaration site
references	List of all usage locations (file, line, column)
signature	For functions: parameter types and return type
is_initialized	Whether the variable has been assigned before use
is_used	Whether the symbol is read anywhere in its scope

5.1.1. Scope Hierarchy

The symbol table is **hierarchical**, mirroring the block structure of the program. Scopes are arranged as a tree; name lookup walks from inner to outer.

```
Global Scope
  [function] 'factorial' : (int) -> int    defined at 1:5
    Block Scope [factorial body]
```

```
[parameter] 'n' : int                defined at 1:23
Block Scope [if-true branch]
  (no new declarations)
```

Listing 2: Scope chain for a C function

5.1.2. Two-Pass Strategy for Forward References

For languages that allow forward references (Java class members, C function prototypes), use a **two-pass approach**:

1. **Pass 1 (Declaration Scan)**: collect all top-level names into the global scope *before* entering any function body.
2. **Pass 2 (Resolution Pass)**: walk function bodies and expression trees, resolving every name reference against the fully populated scope chain.

Example: Symbol Table for a Java class

```
Class Scope [Animal]
[field] 'name' : String (private) defined at 2:20
[field] 'age' : int (private) defined at 3:16
[method] 'speak' : () -> void (public) defined at 5:17
[method] 'getName' : () -> String (public) defined at 8:19
[ctor] 'Animal' : (String, int) -> void defined at 11:12

Class Scope [Dog] extends [Animal]
[field] 'breed' : String (private) defined at 18:20
[method] 'speak' : () -> void (public, @Override) defined at 20:17
[ctor] 'Dog' : (String, int, String) -> void defined at 24:12
```

5.2. Scope Analysis and Name Resolution

When a name is referenced, apply **lexical scope resolution**:

1. Search the innermost (current) scope first.
2. Walk outward through enclosing scopes to the global scope.
3. For OOP languages, also search the class scope and its superclass chain.
4. If not found anywhere: emit an **“undefined symbol”** semantic error with source location.
5. If found in an outer scope but shadowed by an inner declaration: emit a **“variable shadows outer declaration”** warning.

Example: Scope Resolution (Python LEGB Rule)

```

1 x = 10                # Global scope: x -> int(10)
2
3 def outer():
4     x = 20            # Enclosing scope: x -> int(20), shadows global
5     def inner():
6         print(x)      # Resolves to enclosing x = 20, NOT global x
7     inner()
8
9 outer()               # prints 20, not 10

```

The system must correctly resolve the `x` in `inner` to `outer.x`. This demonstrates Python's LEGB rule (Local, Enclosing, Global, Built-in).

5.3. Type System Implementation**5.3.1. Statically-Typed Languages (C, Java, C++, Go)**

Annotate every AST expression node with its computed type and enforce typing rules:

- **Literals:** `42` $\Rightarrow$ `int`; `3.14` $\Rightarrow$ `double`; `"hello"` $\Rightarrow$ `char*` (C) or `String` (Java).
- **Binary expressions:** apply typing rules for each operator, including implicit widening (e.g., `int + double` $\Rightarrow$ `double`).
- **Function calls:** verify argument count and types against the declared signature.
- **Assignments:** verify the right-hand side type is assignable to the left-hand side declared type.
- **Return statements:** verify the returned type matches the enclosing function's return type.

Example: Type Errors Detected in C

```

1 int x    = 3.14;      /* Warning: double -> int loses precision */
2 char *s  = 42;        /* Error: cannot assign int to char*    */
3 int y    = factorial("hello"); /* Error: argument type mismatch:
4                                expected int, got char*        */
5 void foo() { return 5; } /* Error: void function returning a value */

```

5.3.2. Dynamically-Typed Languages (Python, JavaScript)

Implement **flow-sensitive type inference**: track the inferred type of variables through assignments and branching, and use inferred types to guide completion and warn about likely type errors.

Example: Type Inference in Python

```

1 x = 42          # inferred: int  -> offer int attributes
2 x = "hello"     # inferred: str  -> offer str methods (.upper, .split ...)
3 x = [1, 2, 3]   # inferred: list -> offer list methods (.append, .sort ...)
4
5 def greet(name): # 'name' type inferred from call sites
6     return "Hello, " + name # if name is str: ok; else: type warning

```

5.4. Auto-Completion Engine

The central deliverable of Phase 2 is a **context-aware code completion system**. Given a source file and a cursor position (line + column), the system must:

1. **Determine the completion context** from the token preceding the cursor:
 - After . or ->: **member-access** completion.
 - After :: **scope-resolution** completion (C++/Java).
 - At start of statement / after operator: **general scope** completion (all visible symbols).
 - Inside a function argument list: **parameter-type-guided** completion.
2. **Query the Symbol Table** for all symbols visible at the cursor location.
3. **Filter and rank** by prefix match, then fuzzy match.
4. **Return a structured completion list**, each item containing: label, kind, detail (type/signature), and a sortOrder score.

Example: Member-Access Completion in C

```

1 struct Point { int x; int y; };
2 struct Point p = {1, 2};
3 p.|      /* cursor here */

```

The system identifies p as type struct Point and offers:

Label	Kind	Detail
x	Field	int
y	Field	int

Example: Method Completion in Java

```

String s = "hello";
s.|      /* cursor here */

```

The system identifies s as String and offers all public String methods: length(), charAt(int), substring(int, int), toUpperCase(), split(String), ...each with its full signature.

5.5. Diagnostic System

The system must produce **structured, machine-readable diagnostics** for all detected errors:

Phase	Error Class	Severity	Example
Lexer	Unrecognized character	Error	@ in C code
Lexer	Unterminated string literal	Error	"hello without closing "
Parser	Unexpected token	Error	int x = ;
Parser	Missing closing delimiter	Error	if (x { missing)
Semantic	Undefined symbol	Error	Variable y not declared
Semantic	Type mismatch in assignment	Error	int x = "hi"
Semantic	Type mismatch in function call	Error	Wrong argument type
Semantic	Duplicate declaration	Error	Two int x in same scope
Semantic	Wrong number of arguments	Error	foo(1, 2) but foo takes 1 param
Semantic	Return type mismatch	Error	void function returning value
Semantic	Variable shadows outer	Warning	Inner x hides outer x
Semantic	Use before initialization	Warning	Reading x before any assignment
Semantic	Unused variable	Info	Declared but never read

Each diagnostic must include: severity, message, file, line, column, and length (to underline the exact offending span).

5.6. Evaluation Criteria for Phase Two

6. Phase Three: Advanced IDE Intelligence and Program Analysis

Phase 3 — Advanced IDE Features — CFG, Call Graph, Navigation & Refactoring

Phase 3 is the maturity point of the project. The system must now support **program-wide static analysis** and **IDE-grade navigation and refactoring**. These features require building and querying a complete structural model of the entire program. This phase mirrors the *middle-end analysis passes* used in production compilers for optimization, verification, and tooling.

Compiler Design Connection — Program Analysis

Control Flow Graphs (CFGs) and Call Graphs are the primary data structures used in compiler **middle-end optimization** (dead code elimination, function inlining, loop analysis) and **static program verification**. Data-flow analysis — which you study as a *fixed-point computation* over a lattice — runs on CFGs. Go-to-Definition and Find-All-References are powered by the same **reference index** that a compiler's linker builds over object files. Rename Refactoring is a **semantics-preserving transformation**: it must respect scoping rules exactly, which is the same challenge as *alpha-renaming* in lambda calculus — a topic central to the formal semantics of programming languages.

6.1. Control Flow Graph (CFG) Construction

For every function in the program, construct a **Control Flow Graph (CFG)**:

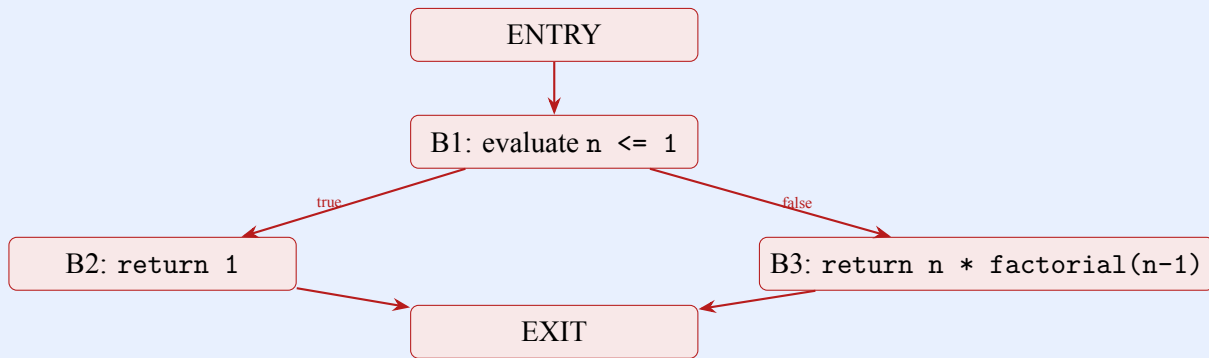
- A **basic block** is a maximal sequence of statements with no branches: control enters at the top and exits at the bottom. It has exactly one entry point and at most two successors.
- **Edges** represent possible execution paths: the true and false branches of conditionals, loop back-edges, and exits via `return`/`break`/`continue`/`throw`.
- Every CFG has a unique **ENTRY** node and one or more **EXIT** nodes.

Example: CFG for factorial

```

1 int factorial(int n) {
2     if (n <= 1) return 1;
3     return n * factorial(n - 1);
4 }

```



6.1.1. CFG-Based Data-Flow Analyses

Using the CFG, implement the following classical analyses:

Definite Assignment Analysis (Forward May-Analysis). For each variable use, verify the variable is **definitely assigned** on every path from ENTRY to that use point. This is a forward data-flow problem: the lattice is $\langle 2^{Vars}, \supseteq \rangle$ (must-analysis), transfer functions set the assigned variable, join is intersection.

Example: Definite Assignment Violation

```

1 int x;
2 if (condition) { x = 42; }
3 printf("%d\n", x); /* Error: x is uninitialized on the path
4                    where 'condition' is false */

```

Live Variable Analysis (Backward May-Analysis). A variable is **live** at a program point if its current value may be used on some future execution path. This is a backward data-flow problem: lattice $\langle 2^{Vars}, \subseteq \rangle$, transfer function removes defined variables and adds used ones, join is union. Used to detect dead assignments.

Unreachable Code Detection. A basic block with no incoming edges (other than ENTRY) is unreachable. Report as a warning. Also detect statements following unconditional return/break/throw within a block.

Example: Unreachable Code Patterns

```

1 int foo() {
2     return 42;
3     printf("never\n"); /* UNREACHABLE: after unconditional return */
4 }
5 void bar(int x) {
6     if (x > 0) {
7         return;
8         x++;           /* UNREACHABLE: after return in if-branch */
9     }
10 }

```

6.2. Call Graph Construction

Build a **program-wide static call graph (CG)**:

- **Nodes**: every function and method definition in the program.
- **Directed edges**: edge $f \rightarrow g$ iff function f contains a call site that resolves to g .
- **Resolution**: use the Symbol Table to resolve each direct call site.
- **Virtual calls** (Java/C++): for polymorphic dispatch, add edges to all methods that could be called based on the declared receiver type and the class hierarchy (conservative over-approximation).

Example: Call Graph (Java polymorphism)

```

main -> Animal.speak      (static type: Animal)
main -> Dog.speak         (via polymorphism: Dog extends Animal)
main -> Cat.speak         (via polymorphism: Cat extends Animal)
Dog.speak -> Animal.speak (explicit super.speak() call)

```

6.2.1. Required Call Graph Queries

Query	Algorithm
Direct callees of function f	Adjacency list lookup
Direct callers of function f	Reverse adjacency list
All transitively reachable callees from f	BFS / DFS from node f
All functions that can reach f	BFS on reversed graph
Detect recursive functions	Cycle detection (DFS with color marking)
Dead functions (not reachable from entry)	Reachability from main node
Strongly connected components	Tarjan's or Kosaraju's algorithm

6.3. Go-to-Definition and Find-All-References

- **Go-to-Definition:** given a cursor position over a symbol usage, return the exact (file, line, column) of its declaration. For overridden methods in OOP, offer both the override and the base-class declaration.
- **Find All References:** given a symbol's definition site, return every (file, line, column) where that symbol is read or written.
- **Hover Information:** given a cursor over any symbol, return its full type signature, its enclosing scope, and any attached documentation comment (Javadoc, Doxygen, or docstring).

Example: Go-to-Definition Response (JSON)

```
Query: goto-definition  file=main.c  line=15  col=12
Response:
{
  "symbol":      "factorial",
  "kind":        "function",
  "type":        "(int) -> int",
  "defined_at":  { "file": "main.c", "line": 1, "col": 5 },
  "references": [
    { "file": "main.c", "line": 15, "col": 12 },
    { "file": "main.c", "line": 16, "col": 24 }
  ]
}
```

6.4. Safe Rename Refactoring

Implement a **semantics-preserving rename operation**:

1. Accept a symbol (identified by its source location) and a new name from the user.
2. **Conflict check:** verify the new name does not already exist in the same scope.
3. **Shadow check:** verify the rename would not accidentally cause shadowing in any inner or outer scope.
4. Produce a **unified diff** listing every line that would be changed.
5. Apply all changes **atomically**: either every occurrence of the symbol is renamed, or none.

Semantic Preservation is Mandatory

The rename must be **scope-aware**. If two variables in different scopes happen to share the same text, only the one the user clicked on may be renamed. A simple text-substitution approach is **not acceptable** and will receive zero credit for this feature.

Example: Rename n to number inside factorial

```

1  /* Before rename:      */      /* After rename:      */
2  int factorial(int n)      int factorial(int number)
3  {                          {
4      if (n <= 1)            if (number <= 1)
5          return 1;          return 1;
6      return n *             return number *
7          factorial(n-1);     factorial(number-1);
8  }                          }
9  /* Other functions that happen to have a variable 'n' are NOT changed. */

```

6.5. Dead Code Detection

Using the CFG and call graph together, identify and report all dead code categories:

- **Unreachable functions:** never reachable from any program entry point.
- **Unreachable basic blocks:** no incoming CFG edges (other than ENTRY).
- **Post-jump statements:** code after return/break/continue/throw.
- **Unused variables:** declared but never read (from liveness analysis).
- **Dead assignments:** a value written to a variable is overwritten before being read (from reaching definitions / liveness).

Example: All Dead Code Categories

```

1  void helper() { }          /* DEAD FUNCTION: never called      */
2
3  int foo() {
4      return 42;
5      int x = 0;             /* UNREACHABLE BLOCK after return */
6  }
7
8  void bar() {
9      int y = compute();
10     y = 99;                 /* DEAD ASSIGNMENT: first value unused */
11     use(y);
12     int z = 1;              /* UNUSED VARIABLE: z never read    */
13 }

```

6.6. Interactive Output Interface

Phase 3 must expose all features through at least one of:

- **Interactive CLI REPL:** accepts commands such as: goto-def main.c 15 12, find-refs factorial, rename factorial compute_fact, show-cfg factorial, callgraph, dead-code.
- **Web UI:** HTML/JavaScript interface to upload source files, browse highlighted code, click symbols for navigation, and visualize the call graph as an interactive diagram.

- **LSP Server:** a Language Server Protocol implementation connectable directly to VS Code or Neovim.

6.7. Evaluation Criteria for Phase Three

7. Bonus Points

Bonus Points

The following features are considered for significant additional credit, ordered roughly by difficulty. Teams are encouraged to select items that connect most naturally to the compiler design topics from the course.

Infrastructure and DevOps

- **Docker Deployment:** containerize the entire system so it runs with a single `docker run` command on any machine. Provide a `Dockerfile` and, if multiple services are used, a `docker-compose.yml`.
- **CI/CD Pipeline:** configure a CI pipeline (GitHub Actions, GitLab CI, etc.) that automatically runs all unit tests on every push, generates highlighted HTML output for a canonical test file, and publishes the report to GitHub Pages. A passing CI badge in the README is required for this bonus.
- **Automated Test Suite with Coverage:** a comprehensive test suite for every module (Lexer, Parser, Semantic Analyzer, CFG builder, Call Graph, IDE features), covering both valid and erroneous programs. Each test specifies the exact expected output. A coverage report demonstrating at least 80% line coverage is expected.

Grammar and Language Extensions

- **Multi-Language Support:** extend the system to support at least one additional programming language using the same Lexer/Parser/Semantic framework. The new language's grammar and token rules must be added as a *plugin* without modifying the shared core. This is the standard design of tools like Tree-sitter.
- **Automatic Language Detection:** implement a classifier that identifies the programming language of an unlabeled source file based on keyword frequencies, delimiter patterns, indentation style, file extensions, and shebang lines. The classifier must output a confidence score for each candidate language.
- **Preprocessor / Macro Expansion (C/C++):** implement a simplified C preprocessor pass (`#define`, `#include`, `#ifdef`/`#endif`) before lexing. Highlight macro expansion sites differently from regular code, and map all error locations back to the pre-expansion source positions.
- **Incremental Re-Parsing:** instead of re-parsing the full file on every change, implement **incremental parsing** that re-parses only the modified region and its syntactic dependents. This is the core technique used by Tree-sitter for real-time IDE responsiveness.

Advanced Compiler Analysis Features

- **Dominator Tree and Post-Dominator Tree:** from the CFG, compute the **dominator tree** (node d dominates n iff every path from ENTRY to n passes through d) using the Lengauer-Tarjan algorithm. Also compute the **post-dominator tree**. Display both alongside the CFG. These are prerequisites for SSA construction and control-dependence computation.
- **Dominance Frontier and SSA Form:** compute the **dominance frontier** of each node (used to determine where to place ϕ -functions), then transform the CFG into **Static Single Assignment (SSA) form** using Cytron et al.'s algorithm. SSA is the intermediate representation used by LLVM, GCC, and virtually all modern optimizing compilers. Displaying the SSA form alongside the AST and original source is a high-value bonus.
- **Reaching Definitions Analysis:** implement the classic **reaching definitions** forward

8. Team Policies

- **Maximum team size is two members**, to ensure optimal coordination and clear task division.
- Members must make **regular, descriptive Git commits** throughout development. The commit history must be traceable, with at least 20 meaningful commits distributed across both members.
- A clear **division of responsibilities** must be documented: who owns the Lexer, Parser, Semantic Analyzer, and so on. Each member must also understand and be able to explain the parts owned by their partner.
- **Up-to-date technical documentation** is mandatory: grammar specification, module design descriptions, algorithm choices and justifications, and a user-facing usage guide.
- Adherence to good software engineering practices: meaningful naming, modular design, separation of concerns, and avoidance of unnecessary global mutable state.
- All source code must be hosted in a **version-controlled repository** (GitHub or GitLab).

9. Project Delivery and Assessment

- Delivery requires the **physical presence of all team members**; each person will be questioned on any component of the system.
- Evaluators provide **test source files**: some syntactically valid, some with lexical or syntactic errors, some with semantic errors. The system must process all of them correctly, applying error recovery where necessary.
- In the presence of errors, the system must **continue processing** the remainder of the file and produce output for all valid portions. Crashing on error is grounds for a significant deduction.
- The **grammar document** (BNF/EBNF) is a required Phase 1 deliverable.
- Complete **technical documentation** must accompany each phase: module architecture, algorithm choices, known limitations, and step-by-step test instructions.
- The system must have at least one **runnable user interface** (CLI or GUI) that accepts input, processes code, and displays all relevant output for the current phase.

10. Conclusion

This project guides you through the construction of a complete **compiler front-end and IDE intelligence layer** built from first principles. Starting from the formal theory of regular languages and context-free grammars, you will implement a system that tokenizes, parses, type-checks, and analyzes real source code — the same pipeline that underlies every production compiler and language server in use today.

By the end of Phase 3 you will have implemented — from scratch — the core components that correspond to the most fundamental concepts in compiler design: DFA-driven lexical analysis, grammar-driven recursive-descent parsing, attribute-grammar-based semantic analysis, control-flow graphs,

iterative data-flow analysis, and call graph construction. These are not abstract academic exercises. They are the algorithms running inside every IDE on every developer's machine, processing millions of lines of code every second.

Good Luck!